

Профессия и план обучения

Кто такой системный аналитик, чем он отличается от смежных ролей, полная карта компетенций и порядок их освоения с проверками на каждом этапе.

Редакция от 06.08.2026

Содержание

1. Кто такой системный аналитик
 - 1.1 Чем занят рабочий день
2. Смежные роли и границы
 - 2.1 Где проходит настоящая граница
 - 2.2 Гибридные вакансии
3. Карта компетенций
 - 3.1 Блок 1. Работа с требованиями
 - 3.2 Блок 2. Документы и артефакты
 - 3.3 Блок 3. Нотации и моделирование
 - 3.4 Блок 4. Базы данных
 - 3.5 Блок 5. API и интеграции
 - 3.6 Блок 6. Архитектура, нефункциональные требования, безопасность
 - 3.7 Инструменты
 - 3.8 Мягкие навыки
 - 3.9 Работа с языковыми моделями
4. План обучения по этапам
 - 4.1 Этап 1. Фундамент требований, три-четыре недели
 - 4.2 Этап 2. Нотации, четыре-шесть недель
 - 4.3 Этап 3. Данные, три-четыре недели
 - 4.4 Этап 4. Интеграции, четыре-шесть недель
 - 4.5 Этап 5. Документы и процессы, три-четыре недели
 - 4.6 Этап 6. Архитектура, нефункциональные требования, безопасность
5. Практика и портфолио
 - 5.1 Полный комплект документации на один пет-проект
 - 5.2 Реверс-анализ работающей системы
 - 5.3 Вклад в открытые проекты
 - 5.4 Переход внутри компании
6. Грейды и состояние рынка
 - 6.1 Что отличает уровни
 - 6.2 Деньги
 - 6.3 Состояние спроса
7. Типичные ошибки новичка
8. Библиотека
 - 8.1 Книги в порядке чтения
 - 8.2 Стандарты

8.3 Что читать регулярно

1. Кто такой системный аналитик

Системный аналитик стоит между теми, кому нужна работающая система, и теми, кто её строит. Его работа начинается там, где заказчик сформулировал желание, и заканчивается там, где разработчик получил постановку, по которой можно писать код без догадок.

Формулировка «переводчик с языка бизнеса на язык разработки» описывает роль неполно. Перевод предполагает, что исходный текст существует и осмыслен. В реальности заказчик приходит с фразой «нам нужно быстрее согласовывать заявки», за которой не стоит ни описания текущего процесса, ни понимания, где именно он тормозит. Работа аналитика начинается на шаг раньше перевода: сначала выяснить, что происходит сейчас, и только потом решать, что должно происходить.

Три вопроса, ответ на которые обязан дать аналитик по любой задаче:

- **Что система должна делать.** Функциональные требования, сценарии, границы, поведение в исключениях.
- **С какими данными она работает.** Сущности, атрибуты, связи, жизненный цикл, ограничения целостности.
- **С чем и как она обменивается.** Внешние системы, контракты, форматы, поведение при недоступности смежника.

Всё остальное в профессии обслуживает эти три вопроса.

1.1 Чем занят рабочий день

Распределение времени на позиции middle в продуктовой команде выглядит примерно так:

Занятие	Доля времени	Что происходит
Разбор и уточнение задач	25–30%	Чтение существующей системы, вопросы заказчику, поиск противоречий
Написание постановок	25%	Описание сценариев, контрактов, моделей данных
Встречи	20%	Обсуждения с заказчиком, груминг, планирование, демо
Поддержка разработки	15%	Ответы на вопросы разработчиков и тестировщиков по ходу реализации
Приёмка	10%	Проверка, что сделано то, что просили

Из таблицы виден неочевидный для новичка факт: написание документов занимает четверть времени, остальное это разговоры и чтение. Аналитик, который любит писать, но не любит выяснять, упирается в потолок быстро.

2. Смежные роли и границы

Границы между аналитическими ролями подвижны, и в вакансиях они смешаны. Понимать разницу нужно для того, чтобы правильно читать описание позиции и не оказаться на работе, которую не ожидал.

Роль	Главный вопрос	Основной артефакт	Кому отвечает
Бизнес-аналитик	Зачем это бизнесу и какой процесс меняем	Модель процесса AS-IS и TO-BE, бизнес-требования	Заказчику
Системный аналитик	Как это должна делать система	Постановка, контракт API, модель данных	Команде разработки
Продуктовый аналитик	Что данные говорят о поведении пользователей	Метрики, гипотезы, результаты экспериментов	Продакту
Аналитик данных	Что происходит по цифрам	Отчёт, дашборд, витрина	Бизнес-заказчику
Архитектор	Из чего система собрана и какой ценой	Архитектурное решение, ADR	Техническому руководству
Продакт-менеджер	Что делаем и в каком порядке	Рoadmap, приоритеты	Бизнесу

2.1 Где проходит настоящая граница

Практическое различие БА и СА видно на одной задаче. Бизнес-аналитик доводит требование до формулировки «заявку должны согласовывать двое ответственных, решение каждого фиксируется отдельно». Системный аналитик обязан продолжить:

- какой метод вызывается при голосовании и что он принимает;
- что происходит, если тот же согласующий голосует повторно;
- можно ли отозвать голос и в каком состоянии заявки;
- что видит заявитель, пока решение не принято;
- какие записи появляются в истории и в журнале аудита;
- что происходит при удалении учётной записи согласующего с незакрытым голосом.

Последний пункт отличает опытного аналитика от начинающего. Основной сценарий описывают все, а половина дефектов приходит из ситуаций, о которых никто не спросил.

2.2 Гибридные вакансии

На рынке часто встречаются совмещённые позиции, и стоит понимать, что за ними стоит:

- **Бизнес-системный аналитик.** Одна роль от разговора с заказчиком до постановки. Типично для средних компаний и интеграторов. Даёт широкий опыт, требует переключения между уровнями абстракции.
- **Системный аналитик и владелец продукта.** Аналитик дополнительно решает, что делать в следующем спринте. Ответственность выше, зона влияния шире.
- **Аналитик и тестировщик.** Встречается в маленьких командах. Риск: приёмка собственной постановки, слепое пятно гарантировано.

3. Карта компетенций

Компетенции удобно держать в виде шести блоков. Каждый разобран в отдельном конспекте комплекта; здесь дан обзор и уровни владения.

3.1 Блок 1. Работа с требованиями

Ядро профессии и то, что отличает аналитика от технического писателя. Уровни требований, методы выявления, критерии качества, приоритизация, управление изменениями, трассируемость. Классификация нефункциональных требований и умение превращать пожелание «должно работать быстро» в проверяемое число.

Уровень для найма: уверенно на входе. Это спрашивают у всех, включая джунов.

3.2 Блок 2. Документы и артефакты

SRS, техническое задание по ГОСТ 34.602, user story с критериями приёмки, use case по Коберну, глоссарий, матрица прав, матрица трассируемости, vision & scope. Умение выбрать формат под ситуацию: гибкой команде не нужен стостраничный документ, а госзаказ не примет доску с карточками.

Уровень для найма: знать структуру основных форматов, уметь написать постановку на среднюю фичу.

3.3 Блок 3. Нотации и моделирование

BPMN для процессов, UML для системы (use case, sequence, activity, state machine, class), ERD для данных, C4 для архитектуры. Не заучивание всех элементов, а понимание, какая нотация описывает какой слой и где границы её применимости.

Уровень для найма: BPMN и три-четыре диаграммы UML уверенно, остальное на уровне узнавания.

3.4 Блок 4. Базы данных

Проектирование модели от концептуальной до физической, нормализация и осознанная денормализация, ключи и связи, транзакции и уровни изоляции, индексы, планы запросов, масштабирование, семейства NoSQL и границы их применения. SQL в объёме сложных выборки с соединениями, агрегатами и оконными функциями.

Уровень для найма: SQL на практическом уровне спрашивают почти всегда, задачу дают прямо на встрече.

3.5 Блок 5. API и интеграции

HTTP и REST, OpenAPI, SOAP, gRPC, GraphQL, брокеры сообщений. Идемпотентность, семантика доставки, повторы, таймауты, компенсации, версионирование контракта. Структура постановки на интеграцию.

Уровень для найма: именно здесь проходит граница между junior и middle. Опыт проектирования интеграций и владение OpenAPI входят в число самых востребованных требований рынка.

3.6 Блок 6. Архитектура, нефункциональные требования, безопасность

Стили архитектуры и их компромиссы, кэш, очереди, отказоустойчивость. Нефункциональные требования в измеримом виде. Аутентификация против авторизации, JWT, OAuth 2.0 и OpenID Connect, модели доступа RBAC и ABAC, персональные данные и 152-ФЗ, базовые уязвимости.

Уровень для найма: от middle и выше. Джуна об этом спросят поверхностно, но ответ «не знаю ничего» насторожит.

3.7 Инструменты

Назначение	Инструменты
Задачи и документация	Jira, YouTrack, Confluence, Notion
Диаграммы	Draw.io , Miro, PlantUML, Mermaid, Camunda Modeler
API	Postman, Swagger Editor, SoapUI, Insomnia
Данные	DBeaver, pgAdmin, любой SQL-клиент
Макеты	Figma (читать и комментировать, не рисовать)
Версионирование	Git на уровне ветки, коммита, пул-реквеста

3.8 Мягкие навыки

Половина работы, и на собеседовании их проверяют не меньше технических:

- **Интервью со стейкхолдером.** Открытые вопросы, техника пяти почему, отделение желания от потребности. Заказчик просит добавить поле в форму, а нужна ему возможность найти заявку по номеру пропуска.
- **Фасилитация.** Довести обсуждение до решения, а не до третьего круга по тем же аргументам.
- **Письменная речь.** Аналитика оценивают по тексту. Двусмысленная формулировка стоит команде недели работы.
- **Разрешение конфликта требований.** Два подразделения хотят противоположного, решение принимать нужно, и оно должно быть обосновано.

- **Умение обосновать отказ.** «Нет» без оценки стоимости и рисков звучит как каприз.
- **Презентация одного решения разным аудиториям.** Бизнесу и разработчикам объясняют разными словами.

3.9 Работа с языковыми моделями

К 2026 году владение инструментами на базе больших языковых моделей перестало быть отличительным преимуществом и вошло в базовые ожидания. Что реально применяют: черновики SQL и спецификаций, разбор объёмной документации, генерация диаграмм в текстовых нотациях, переформулирование требований.

Важная оговорка, которую стоит проговаривать и на собеседовании: инструмент ускоряет черновик, ответственность за корректность постановки остаётся на человеке. Модель уверенно придумывает несуществующие поля и правдоподобно врёт про поведение системы. Проверка фактов по коду и по документации остаётся ручной работой.

4. План обучения по этапам

План рассчитан на человека с техническим бэкграундом, который уже знает SQL, HTTP и устройство приложений. Без такой базы каждый этап занимает примерно вдвое больше времени.

4.1 Этап 1. Фундамент требований, три-четыре недели

Уровни и типы требований, методы выявления, критерии качества, приоритизация. Основной источник: Вигерс и Битти.

Проверка результата. Взять знакомую систему, написать на неё vision & scope и десяток функциональных требований. Каждое требование проверить на однозначность: если два человека прочтут по-разному, требование не готово.

4.2 Этап 2. Нотации, четыре-шесть недель

Сначала BPMN, потом UML, потом ERD. Сразу писать в текстовых нотациях: диаграмма в PlantUML или Mermaid лежит рядом с кодом, ревьюится и не протухает молча, в отличие от картинки, которую однажды нарисовали мышью.

Проверка результата. Смоделировать сквозной процесс из полутора десятков шагов с ветвлениями и исключениями в BPMN, затем тот же сценарий в sequence-диаграмме. Если диаграммы получились одинаковыми, разница между нотациями ещё не понята.

4.3 Этап 3. Данные, три-четыре недели

Концептуальная, логическая и физическая модель. Нормализация и её осознанное нарушение. Ключи, связи, целостность. Транзакции и изоляция. Индексы и планы запросов.

Проверка результата. Спроектировать модель на пятнадцать-двадцать таблиц под реальный домен и вслух обосновать каждое решение о нормализации и каждый индекс.

4.4 Этап 4. Интеграции, четыре-шесть недель

Самый важный этап для трудоустройства. REST и OpenAPI руками, SOAP на уровне чтения WSDL, брокеры сообщений, идемпотентность, повторы, компенсирующие сценарии.

Проверка результата. Написать спецификацию OpenAPI на десяток методов с примерами запросов, ответов и ошибок. Отдельно написать постановку на интеграцию двух систем: маппинг полей, поведение при недоступности смежника, что происходит при повторной доставке сообщения.

4.5 Этап 5. Документы и процессы, три-четыре недели

SRS, структура ГОСТ 34.602, user story с критериями приёмки, Scrum и место аналитика в спринте, работа в Jira и Confluence.

Проверка результата. Свести всё сделанное на предыдущих этапах в один связный комплект документации по пет-проекту: от границ системы до контрактов и модели данных.

4.6 Этап 6. Архитектура, нефункциональные требования, безопасность

Изучается медленно и в основном на реальных задачах. Читается параллельно с работой, а не блоком.

Итого: четыре-шесть месяцев плотных занятий до состояния, когда можно идти на собеседования, при условии, что техническая база уже есть.

5. Практика и портфолио

Главная проблема входа в профессию: требований не бывает без заказчика, а заказчика не бывает без работы. Обходные пути, в порядке убывания пользы.

5.1 Полный комплект документации на один пет-проект

Один сквозной домен, доведённый до конца, весит в резюме больше, чем пять пройденных курсов. Состав комплекта:

1. Границы системы и краткое описание задачи.
2. Глоссарий предметной области.
3. Модель процесса в BPMN, состояние AS-IS и TO-BE.
4. Два-три use case с альтернативными потоками и исключениями.
5. ER-диаграмма и описание сущностей.
6. Спецификация API в OpenAPI.
7. Матрица ролей и прав.
8. Нефункциональные требования в измеримом виде.
9. User stories с критериями приёмки на один спринт.

Домен лучше брать нескучный и знакомый: бюро пропусков, запись к врачу, склад, доставка. Знакомый домен позволяет заметить исключения, которых нет в учебниках.

5.2 Реверс-анализ работающей системы

Взять продукт с открытым API и восстановить по нему модель данных, процессы и границы. Заодно тренируется навык, на который уходит большая часть рабочего времени: разбор чужой системы без документации.

5.3 Вклад в открытые проекты

У открытых проектов документация обычно слабое место, и осмысленный вклад заметен. Заодно появляется публичная история работы.

5.4 Переход внутри компании

Для действующего разработчика это самый короткий путь. Взять на себя постановки и документацию в своей же команде, накопить полгода фактического опыта и перейти формально. Такой переход проходит без просадки в грейде, в отличие от входа с рынка с нуля.

6. Грейды и состояние рынка

6.1 Что отличает уровни

Уровень	Признаки
Junior	Понимает нотации, пишет постановку по образцу, задачи приходят разобранными, работает под присмотром
Middle	Ведёт фичу от разговора с заказчиком до приёмки, проектирует интеграции, спорит с архитектором по существу, сам находит недостающие требования
Senior	Отвечает за домен целиком, находит противоречия до старта разработки, влияет на архитектурные решения, растит младших, ведёт переговоры со смежными командами
Lead	Отвечает за процесс анализа в подразделении, нанимает и развивает аналитиков, отвечает за качество постановок командой

Переход из junior в middle определяется одним признаком: способностью самостоятельно довести незнакомую задачу до постановки, задав правильные вопросы, а не дождавшись, пока их задаст кто-то другой.

6.2 Деньги

Данные агрегаторов расходятся в разы, поэтому цифры ниже стоит считать ориентиром, а не фактом. По совокупности источников на 2026 год: junior примерно 60–120 тысяч, middle 160–240 тысяч, senior от 250 тысяч и выше. Один агрегатор даёт медиану junior около 62 тысяч против middle около 227 тысяч, и такой разрыв говорит скорее о качестве выборки, чем о рынке.

Существенно влияют на цифру отрасль и тип компании: банк и финтех платят заметно больше, чем системный интегратор или региональная компания. Проверять стоит по открытым вакансиям в своём регионе и стеке, а не по обзорам.

6.3 Состояние спроса

Массовый набор джунов свернулся, спрос сместился в точечный поиск специалистов уровня middle и senior. Для входа без опыта это плохая новость. Для перехода из разработки хорошая: технический бэкграунд закрывает ровно ту часть, из-за которой перестали брать новичков, а именно способность разговаривать с командой на её языке и понимать цену реализации.

7. Типичные ошибки новичка

Проектировать решение вместо выяснения задачи. Профессиональная болезнь пришедших из разработки. Услышал требование, сразу придумал таблицу и метод. Проблема в том, что настоящая задача часто оказывается другой, и работа выбрасывается целиком.

Записывать желания заказчика буквально. Заказчик описывает решение, а не потребность. Просьба «сделайте выгрузку в Excel каждый понедельник» может означать «мне нужно видеть недельную сводку», и тогда дашборд решает задачу лучше.

Писать требования, которые нельзя проверить. «Интерфейс должен быть удобным», «система должна работать стабильно». Такое требование не может быть ни выполнено, ни нарушено.

Описывать только основной сценарий. Счастливый путь пишут все. Дефекты приходят из отказов, повторных нажатий, потери связи, одновременной работы двух пользователей и удаления связанных данных.

Прятаться за документом. Отправить постановку в чат и считать работу сделанной. Разработчик её не прочтёт, если она на двадцать страниц, а тестировщик прочтёт иначе. Постановку нужно проговаривать.

Молчать о противоречиях. Два требования конфликтуют, аналитик заметил и решил, что разберутся сами. Не разберутся; конфликт всплывёт на приёмке, когда переделывать дорого.

Не вести историю решений. Через полгода никто не помнит, почему сделали именно так, и решение переигрывают заново, наступая на те же грабли. Короткая запись «рассматривали такой вариант, отказались по такой причине» экономит недели.

Учить нотации как самоцель. Знать все элементы BPMN и не уметь выяснить у заказчика, что происходит при отказе, это диплом без профессии.

8. Библиотека

8.1 Книги в порядке чтения

1. **Карл Вигерс, Джой Битти. Разработка требований к программному обеспечению.** Основной учебник профессии. Читать первой и возвращаться.
2. **Алистер Коберн. Современные методы описания функциональных требований к системам.** Use cases от автора метода.
3. **Дин Леффингуэлл, Дон Уидриг. Принципы работы с требованиями к ПО.** Процессная сторона работы с требованиями.
4. **Мартин Фаулер. UML. Основы.** Короткая и по делу, без академизма.
5. **Мартин Клеппман. Высоконагруженные приложения.** Для нефункциональных требований, интеграций и понимания того, о чём спорят архитекторы.
6. **ВАВОК.** Свод знаний по бизнес-анализу, как справочник, а не для чтения подряд.

8.2 Стандарты

Документ	О чём
ГОСТ 34.602	Техническое задание на создание автоматизированной системы
ГОСТ 19 (ЕСПД)	Единая система программной документации
ISO/IEC/IEEE 29148	Инженерия требований, современная замена IEEE 830
ISO/IEC 25010	Модель качества программных систем, основа для нефункциональных требований
BPMN 2.0	Спецификация нотации моделирования процессов
UML 2.5	Спецификация языка моделирования

Стандарты ГОСТ обязательны, если целишься в государственный сектор, банки, оборонные предприятия и системных интеграторов. В продуктовых компаниях они почти не встречаются.

8.3 Что читать регулярно

Профильные разделы Хабра по системному анализу, телеграм-сообщества аналитиков, разборы вакансий. Полезнее всего читать не советы, а чужие постановки и спецификации: качество работы видно только на примерах.